

# Rock–Paper–Scissors CNN

## Contents

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                     | <b>2</b>  |
| 1.1      | Problem statement . . . . .                                             | 2         |
| 1.2      | Objectives . . . . .                                                    | 2         |
| 1.3      | Constraints and limitations . . . . .                                   | 2         |
| 1.4      | Overview of the three models . . . . .                                  | 2         |
| <b>2</b> | <b>Dataset preparation</b>                                              | <b>3</b>  |
| 2.1      | Original dataset overview . . . . .                                     | 3         |
| 2.2      | Split strategy . . . . .                                                | 3         |
| 2.3      | Augmentation policies . . . . .                                         | 4         |
| <b>3</b> | <b>CNN architectures</b>                                                | <b>5</b>  |
| 3.1      | CNNs overview . . . . .                                                 | 5         |
| 3.2      | Tiny CNN . . . . .                                                      | 5         |
| 3.3      | Small CNN . . . . .                                                     | 6         |
| 3.4      | Medium Base CNN (template for tuning) . . . . .                         | 6         |
| 3.5      | Hyperparameter tuning, cross-validation, and final Medium CNN . . . . . | 6         |
| <b>4</b> | <b>Models evaluation</b>                                                | <b>7</b>  |
| 4.1      | Performance metrics and training curves . . . . .                       | 7         |
| 4.2      | Visual examples of errors . . . . .                                     | 10        |
| 4.2.1    | Tiny CNN . . . . .                                                      | 10        |
| 4.2.2    | Small CNN . . . . .                                                     | 11        |
| 4.2.3    | Medium CNN . . . . .                                                    | 11        |
| 4.3      | Models comparison . . . . .                                             | 12        |
| <b>5</b> | <b>Generalised inference</b>                                            | <b>12</b> |
| 5.1      | Description of own images . . . . .                                     | 12        |
| 5.2      | Description of results . . . . .                                        | 12        |

# 1 Introduction

## 1.1 Problem statement

The goal of this project is to design, train, and evaluate Convolutional Neural Networks capable of recognising hand gestures corresponding to the “rock–paper–scissors” game. The task is a standard image classification problem, where the model must correctly predict one of three possible classes.

This problem is particularly well suited to convolutional architectures, as these models are highly effective at learning spatial hierarchies and local features such as edges, shapes, and textures directly from image data. The challenge lies in balancing accuracy, generalization, and computational efficiency within the constraints of limited training data and hardware resources.

## 1.2 Objectives

The main objectives of this work are to explore and compare multiple CNN architectures of increasing complexity, which use different levels of image augmentation to improve generalisation and robustness, and to successfully enhance a model by performing hyperparameter tuning using a systematic cross-validation approach to identify optimal configurations.

The final objective is to evaluate each model using standard performance metrics such as accuracy, precision, recall, and F1-score, and to test the final model’s generalization ability on custom, real-world images captured outside the dataset distribution.

## 1.3 Constraints and limitations

While CNNs are powerful models, their performance is constrained by computational resources, data quality, and most importantly a time limitation; in this project, several practical limitations influenced design decisions to account for such limitations.

To limit the running time, deeper CNNs with many parameters were avoided to maintain reasonable training times on a standard CPU/GPU. Moreover, the architectures were deliberately kept relatively small to emphasize interpretability and efficiency rather than maximizing absolute accuracy. These constraints guided the choice of restricting input resolution ( $128 \times 128$ ), moderate depth, and lightweight training configurations. The limitations connected to the dataset is that the background, lighting, and hand orientations don’t vary significantly across samples, which can hinder generalization to new conditions; the augmentations applied in each model try to account for these issues.

## 1.4 Overview of the three models

Three CNN architectures were implemented, each representing a progressive increase in depth, capacity, and augmentation complexity.

**Tiny CNN:** a compact baseline model with two convolutional stages. It serves as a lightweight reference to evaluate how a minimal CNN performs on this classification task. Basic geometric augmentations were applied, such as horizontal flip, small rotations and zooms, and contrast adjustments, to increase variability in the dataset and consequently enhance the model’s generalisation capacity.

**Small CNN:** a deeper variant with additional convolutional layers, designed to capture more complex spatial hierarchies; it employs RandAugment, a data augmentation technique that applies random combinations of geometric and photometric transformations. This model aims to further improve generalization compared to the Tiny CNN while maintaining moderate training time.

**Medium CNN:** the most complex architecture, featuring three convolutional stages with increased filter counts and dropout regularization to mitigate overfitting. It uses a custom

“green strong” augmentation strategy that includes background replacement, rotations, and colour perturbations in order to perform better on images external to the test set belonging to the original dataset. Furthermore, this model undergoes hyperparameter tuning with 3-fold cross-validation, testing variations in learning rate and dropout to identify the configuration with the best validation performance.

## 2 Dataset preparation

### 2.1 Original dataset overview

The dataset used in this project is the Rock–Paper–Scissors image dataset, containing photographs of hands forming one of three gestures. Each class folder holds a set of labelled RGB images in .jpg or .png format. These images don’t exhibit significant variability in lighting, orientation, background color, and skin tone, which provides an easier classification scenario on the test set coming from the same dataset (Figure 1), but could train models which don’t generalise well on data coming from other sources.

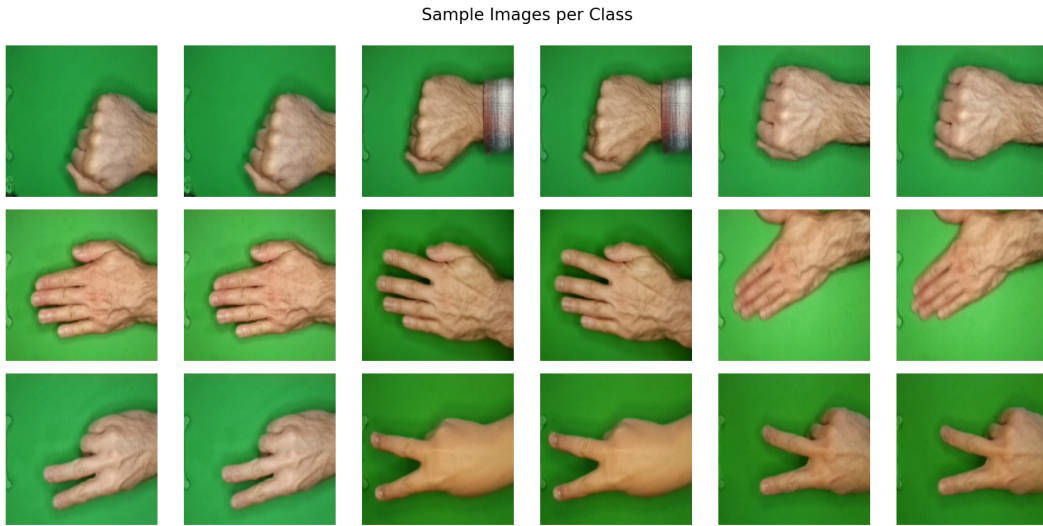


Figure 1: Sample grid.

The dataset is relatively small compared to large-scale benchmarks, making it ideal for exploring how data augmentation and model regularization affect generalization. The classes are evenly distributed (Figure 2), so adding weights to each class was not necessary. Before training, all images were standardized to a resolution of  $128 \times 128 \times 3$  pixels and normalized to a  $[0,1]$  range to stabilize training and ensure consistent input across all models.

### 2.2 Split strategy

To ensure a reliable and unbiased evaluation, the dataset was divided into three separate subsets. The training set, used to fit model parameters, contained 70% of the original dataset. In order to train the models, 15% of the original dataset became the validation set, used for early stopping, accuracy and loss evaluation, and hyperparameter tuning. The test set, another 15%, was reserved exclusively for final model evaluation and reporting. The splits were created using stratified sampling, maintaining equal class proportions across all subsets.

The following process was applied sequentially:

1. A first split separates the test set from the rest of the data.
2. The remaining data is then divided again to extract a validation set. This guarantees that

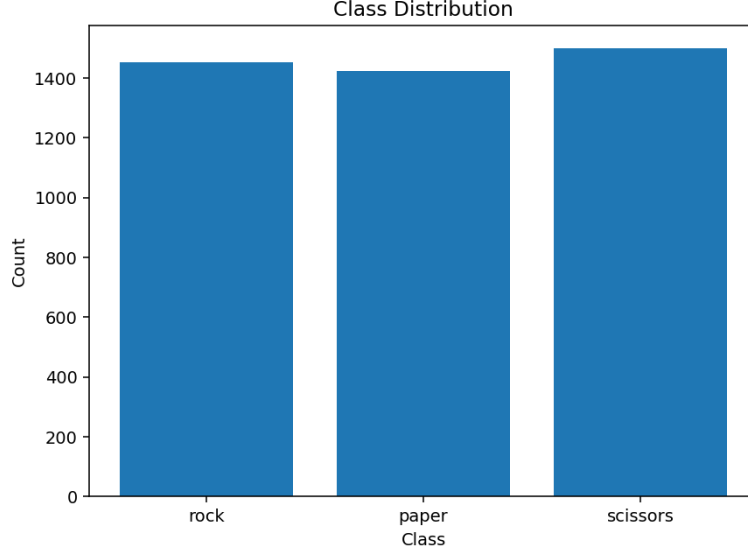


Figure 2: Class distribution.

no overlap exists between training, validation, and test data.

A fixed random seed (seed = 42) was used throughout all splitting and shuffling operations to ensure reproducibility of the results.

### 2.3 Augmentation policies

Since the dataset lacks significant diversification, data augmentation plays a crucial role in preventing overfitting and improving model generalization. Three distinct augmentation strategies were developed and assigned to the models according to their complexity level:

**Basic augmentation, used in Tiny CNN:** it applies a small set of common image transformations implemented through Keras preprocessing layers, like random horizontal flipping, small random rotations ( $\pm 10\%$ ), random zoom ( $\pm 15\%$ ), and random contrast adjustments ( $\pm 15\%$ ). These transformations simulate minor variations in viewpoint and lighting, which helps the model become more robust without altering the semantic meaning of the gestures.

**RandAugment, used in Small CNN:** a stronger and more diverse policy, introduced by Google Research (<https://colab.research.google.com/github/keras-team/keras-io/blob/master/examples/vision/ipynb/randaugment.ipynb>), automatically applies  $N$  random transformations per image, each with a randomly chosen magnitude  $M$ ; unlike traditional fixed augmentations, RandAugment dynamically explores combinations of operations such as rotation, shear, translation, colour jittering, brightness and contrast modifications. In this project, RandAugment was configured using KerasCV with parameters  $N = 2$  and  $M = 0.3$ ; this ensures that each image undergoes two random perturbations at moderate intensity, exposing the network to a wide distribution of visual variations.

**Green strong augmentation, used in Medium Base CNN:** an aggressive augmentation policy, specifically designed for the dataset, as the images feature green-screen or uniform backgrounds. The process involves two main steps:

1. Green background removal: Pixels within the HSV color range corresponding to green ( $[35, 40, 40]$  to  $[85, 255, 255]$ ) are detected and replaced with pure white, eliminating bias from background color.

2. Photometric and geometric perturbations: Random horizontal flips with a 50% probability, random rotations up to  $\pm 25^\circ$ , brightness and contrast variations up to  $\pm 30\%$ .

This combined process produces a diverse set of visually distinct samples while retaining the original gesture semantics; it helps the CNN learn to focus on hand shape and pose rather than on irrelevant background textures.

Overall, the augmentation pipeline provides three progressively stronger data transformation strategies:

| Model      | Augmentation Type | Description                                      |
|------------|-------------------|--------------------------------------------------|
| Tiny CNN   | Basic             | Small geometric and contrast perturbations       |
| Small CNN  | RandAugment       | Randomized transformations with varying strength |
| Medium CNN | Green Strong      | Background removal + strong photo augmentations  |

Table 1: Augmentation strategies.

### 3 CNN architectures

#### 3.1 CNNs overview

Convolutional Neural Networks were selected for this task because of their ability to automatically extract spatial features from images.

Three progressively complex architectures were developed following a unified design philosophy: fully convolutional blocks with ReLU activations, max pooling for spatial downsampling, and dropout for regularization.

Each model is built through a modular helper function that constructs the network from a sequence of stages. Each stage contains one or more  $3 \times 3$  convolutional layers with “same” padding and ReLU activation, optionally followed by a max-pooling layer. After the final stage, Global Average Pooling replaces flattening to minimize parameter count, followed by a dropout layer and a dense softmax classifier with three outputs. All models use the Adam optimizer with categorical cross-entropy loss and label smoothing to improve generalization by reducing overconfidence on ambiguous samples. This design ensures a consistent framework that scales in depth, filter count, and regularization strength across models.

#### 3.2 Tiny CNN

The Tiny CNN serves as a lightweight baseline model, designed to test the feasibility of the task with minimal computational cost while preserving CNN fundamentals. The network consists of:

- one convolutional layers with 16 filters, followed by max pooling;
- one convolutional layer with 32 filters;
- global average pooling, dropout (0.2), and a 3-unit softmax output layer.

Training was performed for 15 epochs with a learning rate of 0.001 and basic data augmentation. This model trains rapidly and provides a simple yet effective benchmark, though its small number of parameters limits its ability to capture complex visual patterns.

### 3.3 Small CNN

The Small CNN increases architectural depth by introducing an additional convolutional block and more filters per stage, enabling the model to capture finer spatial details such as finger contours and gesture textures.

The network consists of:

- two convolutional layers with 32 filters each, followed by max pooling;
- one convolutional layer with 64 filters;
- global average pooling, dropout (0.3), and a 3-unit softmax output layer.

Training used 15 epochs, a learning rate of 0.001, and RandAugment for data augmentation. By balancing depth and efficiency, this model achieves higher accuracy and improved generalization compared to the Tiny CNN, while maintaining moderate training time.

### 3.4 Medium Base CNN (template for tuning)

The Medium Base CNN acts as a template architecture used for hyperparameter tuning, and it represents the most complex architecture in this project, extending the Small CNN with deeper convolutional stages, stronger regularization, and more advanced augmentation.

The network consists of:

- two convolutional layers with 32 filters each, followed by max pooling;
- two convolutional layers with 64 filters each, followed by max pooling;
- one convolutional layer with 128 filters;
- global average pooling, dropout (0.4), and a 3-unit softmax output layer.

### 3.5 Hyperparameter tuning, cross-validation, and final Medium CNN

To obtain the final model, the Medium CNN, a three-fold cross-validation procedure was applied on the Medium Base template to identify the best combination of learning rate, dropout, and batch size.

Each trial trained for approximately six epochs with early stopping (patience = 2) to accelerate evaluation. The following parameters were explored:

- learning rate: [0.0005, 0.001];
- dropout: [0.4, 0.5];
- batch size: [32].

For each configuration, two folds were used for training and one for validation, rotating across all folds. The combination achieving the highest mean validation accuracy was then retrained on the combined training and validation data for 15 epochs using the green\_strong augmentation before testing.

This process produced the Medium CNN, the final and most optimized model in the project; by tuning key hyperparameters rather than merely increasing architectural depth, this approach achieved the best balance between accuracy, robustness, and computational efficiency while preserving a fully unseen test set for final evaluation

| Model      | Conv Layers   | Dropout | Augmentation | Learning Rate |
|------------|---------------|---------|--------------|---------------|
| Tiny CNN   | 2             | 0.2     | Basic        | 0.001         |
| Small CNN  | 3 (2 + 1)     | 0.3     | RandAugment  | 0.001         |
| Medium CNN | 5 (2 + 2 + 1) | 0.4–0.5 | Green Strong | 0.0005–0.001  |

Table 2: Summary of model architectures.

## 4 Models evaluation

### 4.1 Performance metrics and training curves

The performance of all models was evaluated on the held-out test set, using:

- accuracy, which reflects the overall proportion of correctly classified images;
- precision, which measures the proportion of correct positive predictions per class;
- recall, which quantifies how well each model captures all true positives;
- F1-score, which balances precision and recall into a single measure of model robustness.

**Tiny CNN** The Tiny CNN served as the baseline model with only two convolutional stages (16 and 32 filters). Its learning curves (Figure 3) show slow learning and limited capacity, with training accuracy plateauing around 0.53 and validation fluctuating near 0.6. Both losses steadily decrease but remain high, indicating the model is too simple to capture the dataset’s visual complexity.

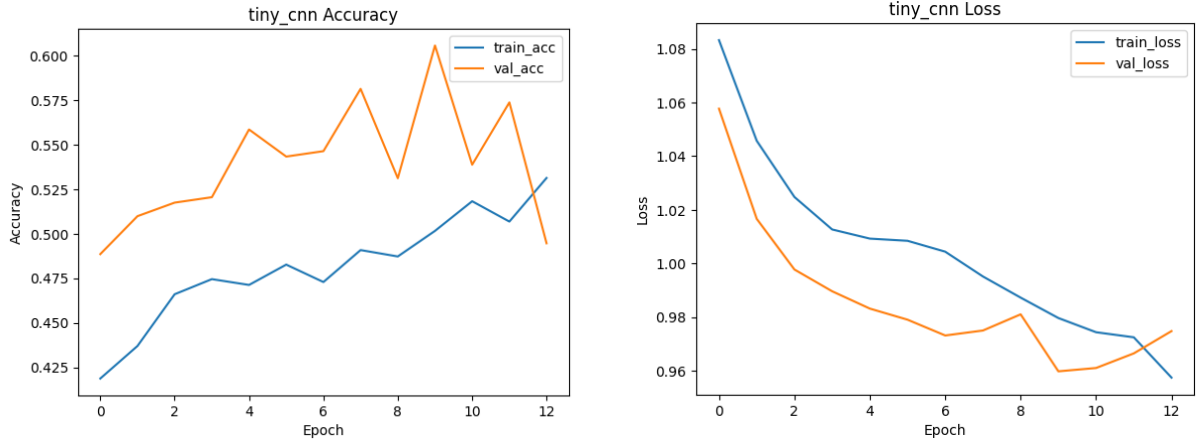


Figure 3: Tiny CNN accuracy and loss curves.

The confusion matrix (Figure 4) confirms this, as the model struggles especially to distinguish between rock and paper, while performing slightly better on scissors: this suggests that low-level spatial filters fail to separate the more visually similar classes.

#### Metrics summary:

Accuracy: 0.589

Precision: 0.610

Recall: 0.589

F1-score: 0.563

Training time: 59.0s

Overall, the Tiny CNN establishes a clear baseline but is too simple to capture the dataset’s

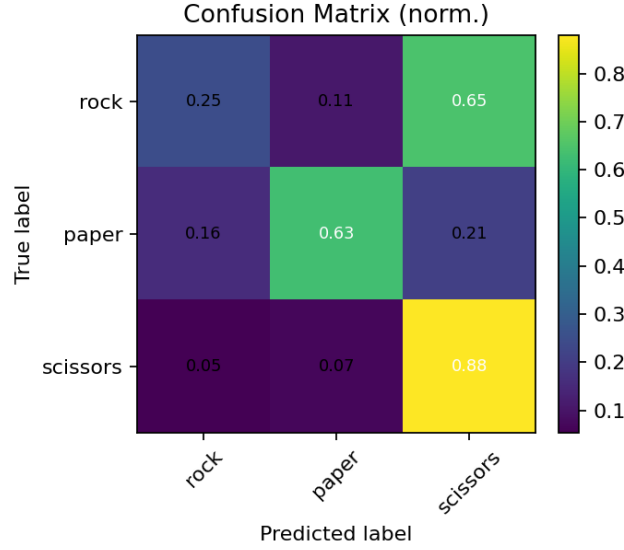


Figure 4: Tiny CNN confusion matrix.

visual complexity. Its low F1-score indicates that both false positives and false negatives occur frequently, highlighting weak discriminative ability.

**Small CNN** The Small CNN introduced additional convolutional depth and a stronger augmentation strategy. The learning curves (Figure 5) show rapid and stable improvement, with validation accuracy rising above 0.8 and training accuracy steadily increasing, evidence of effective learning and good generalization. Both training and validation losses decrease smoothly without divergence, indicating no overfitting and confirming that the model’s added depth and stronger augmentation significantly improve performance compared to the Tiny CNN.

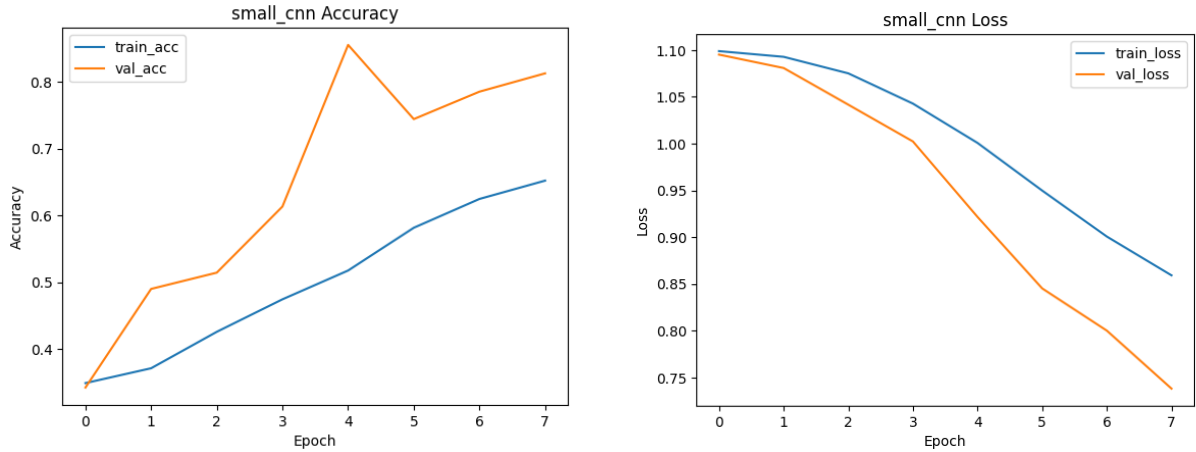


Figure 5: Small CNN accuracy and loss curves.

The confusion matrix (Figure 6) displays diagonal dominance, with only minor confusion between classes, mainly between paper and rock. This shows that the deeper architecture captures more distinctive shape and texture features.

**Metrics summary:**

Accuracy: 0.846

Precision: 0.847

Recall: 0.846

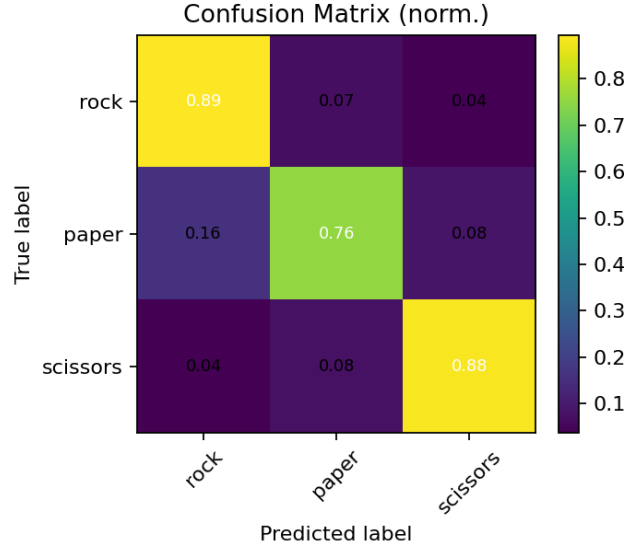


Figure 6: Small CNN confusion matrix.

F1-score: 0.845

Training time: 204.8s

Compared to the Tiny CNN, this model demonstrates a large improvement in all metrics. Its balanced precision and recall confirm consistent performance across all gesture classes. The F1-score near 0.85 suggests that both false positives and false negatives were significantly reduced through better feature extraction and augmented diversity.

**Medium CNN (hyperparameter tuned)** The Medium CNN represents the final and most advanced model in this project; it was derived from the Medium Base CNN template via 3-fold cross-validation, which optimized the learning rate, dropout, and batch size. The best configuration (learning rate = 0.001, dropout = 0.4, batch size = 32) was then retrained on the combined training + validation set using the green\_strong augmentation strategy. Since the model was retrained on all available data after cross-validation, only the training curves are reported (Figure 7), and they show rapid and stable convergence with minimal oscillation, confirming that the tuned hyperparameters provided balanced learning dynamics and strong generalization.

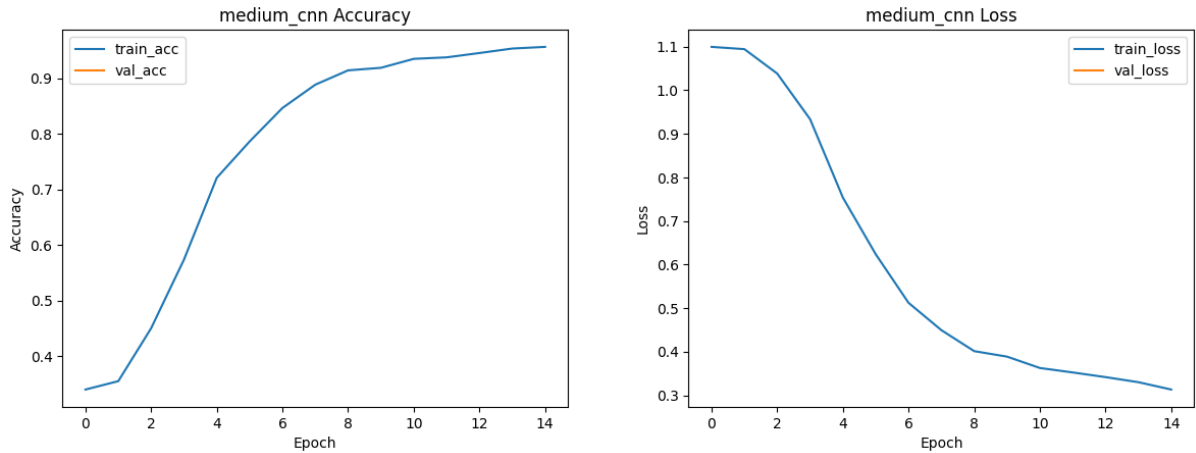


Figure 7: Medium CNN curves (trained on train+val).

The confusion matrix (Figure 8) exhibits nearly perfect class separation, indicating that the model correctly generalizes across lighting, background, and orientation variations. Misclassifications are minimal, confirming stable convergence and robustness.

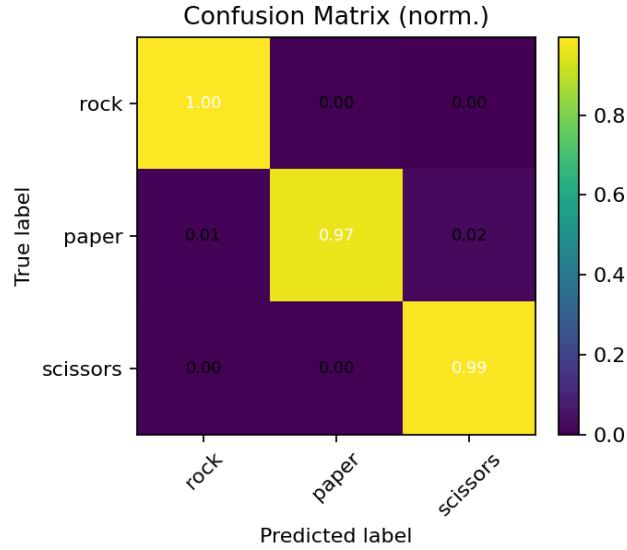


Figure 8: Medium CNN confusion matrix.

#### Metrics summary:

Accuracy: 0.985

Precision: 0.985

Recall: 0.985

F1-score: 0.985

Training time: not measured precisely due to cross-validation overhead, around 1 hour.

This tuned configuration achieves the best overall performance, demonstrating that systematic hyperparameter optimization yields greater benefits than architectural expansion alone. Its F1-score approaching 1.0 indicates exceptional balance between precision and recall, implying that nearly all gestures are correctly identified with minimal confusion between classes.

## 4.2 Visual examples of errors

### 4.2.1 Tiny CNN

The Tiny CNN exhibits several systematic misclassifications, consistent with its underfitting behaviour. It often confuses visually similar gestures or those affected by incomplete framing or low contrast. The following examples illustrate typical failure cases. These examples are

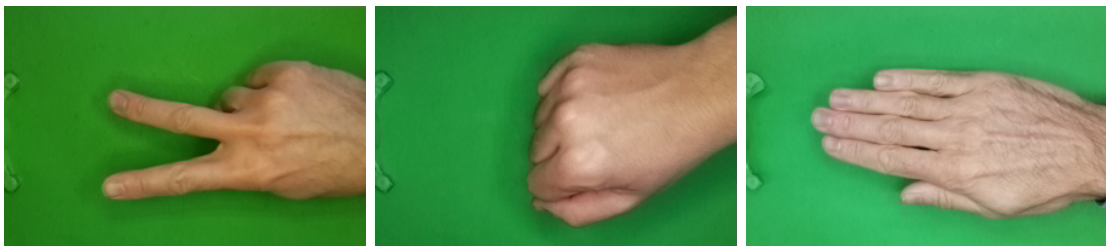


Figure 9: Tiny CNN misclassified examples: (left) scissors predicted as rock; (middle) rock predicted as paper; (right) paper predicted as rock.

visually clear and well representative of their gestures, yet the model still misclassifies them, showing its limited recognition ability.

#### 4.2.2 Small CNN

The Small CNN performs considerably better than the Tiny CNN but still shows occasional misclassifications in challenging situations. Typical errors occur when gesture scale, framing, or angle deviate from the dataset distribution. In the first image, the scissors gesture was mistaken



Figure 10: Small CNN misclassified examples: (left) scissors predicted as paper; (middle) rock predicted as paper; (right) paper predicted as rock.

for paper because the fingers appear extended and the silhouette resembles an open hand. In the second, the rock gesture was predicted as paper, the hand appears unusually large in the frame, making it visually closer to an open palm. Finally, in the third case, the paper gesture was classified as rock since the image is cropped below the knuckles, hiding the lower part of the hand. These examples highlight how gesture scale and cropping strongly influence classification, even when the model generalizes well overall.

#### 4.2.3 Medium CNN

The Medium CNN shows exceptional robustness and only a few isolated errors, generally occurring in atypical conditions such as partial occlusion, extreme zoom, or motion blur. These examples demonstrate that the remaining errors are not due to model weakness but rather image ambiguity. In the first case, the hand showing scissors is partially cut away, making the



Figure 11: Medium CNN misclassified examples: (left) scissors predicted as rock; (middle) rock predicted as paper; (right) paper predicted as scissors.

gesture ambiguous and difficult to interpret. In the second, a rock gesture was misclassified as paper because the hand appears very large, with the fingers not fully bent inward, visually resembling an open palm. In the third image, the paper gesture was classified as scissors since the hand is cropped below the knuckles, the thumb is partly missing from the frame, and the image is slightly blurred. Such misclassifications occur only in visually unclear contexts, confirming that the Medium CNN has achieved strong robustness to normal lighting, background, and scale variations.

### 4.3 Models comparison

The results clearly show that increasing architectural complexity and applying stronger augmentation improve performance substantially.

- The Tiny CNN underperforms severely due to its limited capacity and shallow feature extraction, reflected by its low precision and recall values.
- The Small CNN achieves a strong balance between generalization and efficiency, benefiting most from RandAugment, which provides diverse input variation and reduces overfitting.
- The Medium CNN, optimized through cross-validation, attains near-perfect accuracy and precision, demonstrating that tuning hyperparameters such as learning rate and dropout can yield substantial performance gains without increasing model depth.

Overall, the progression from Tiny to Small to Medium CNNs highlights a clear trade-off between complexity, computation, and generalization. The F1-score trend ( $0.563 \rightarrow 0.845 \rightarrow 0.985$ ) encapsulates this improvement, showing how the refined architecture and systematic tuning nearly eliminate both false positives and false negatives. The Small CNN offers the best compromise between performance and computational efficiency, while the Medium CNN serves as the definitive, high-accuracy model suitable for deployment and real-world evaluation.

| Model      | Params | Accuracy | Precision | Recall | F1-score |
|------------|--------|----------|-----------|--------|----------|
| Tiny CNN   | 5187   | 0.589    | 0.610     | 0.589  | 0.563    |
| Small CNN  | 28835  | 0.846    | 0.847     | 0.846  | 0.845    |
| Medium CNN | 139811 | 0.985    | 0.985     | 0.985  | 0.985    |

Table 3: Model comparison.

## 5 Generalised inference

### 5.1 Description of own images

To evaluate the model’s ability to generalize beyond the controlled dataset, a small set of 12 custom test images was collected, captured under natural lighting conditions and with a uniform white background, with different zoom levels and hand positioning, introducing differences in scale and framing that test the robustness of the trained model. The set includes examples of all three gestures (rock, paper, and scissors) with each class represented equally, with 4 pictures per class.

### 5.2 Description of results

Inference was performed using the final Medium CNN, which had achieved the highest accuracy and generalization performance on the benchmark test set. Out of the 12 custom images, 10 were correctly classified, corresponding to an overall accuracy of approximately 83%. The correctly identified gestures included:

- 2 rock;
- 4 paper;
- 4 scissors;

The two misclassifications occurred for rock gestures, which were incorrectly predicted as scissors. This pattern suggests that the model’s decision boundaries between rock and scissors remain somewhat sensitive to shape variation and zoom level, particularly when the hand occupies a smaller portion of the frame. Overall, these results indicate that the Medium CNN retains strong generalization capabilities; minor errors primarily arise from differences in hand position and proportion, suggesting that incorporating further scale or context-invariant augmentations could further improve robustness in real-world applications.

*I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.*